

AdvisoryAI Hack-to-Hire: Challenge Briefs

The Context: Life as a UK Financial Advisor

Before diving into the challenges, understand who you're building for.

A typical UK Independent Financial Advisor (IFA) manages 150-250 client relationships. They work within strict regulatory frameworks set by the Financial Conduct Authority (FCA), including Consumer Duty regulations introduced in 2023 that require demonstrating ongoing value to clients.

Their day is overwhelmingly reactive: responding to client emails, fielding calls, chasing paperwork, and drowning in administrative tasks. The irony? They became advisors to advise people—yet most spend 60-70% of their time on admin, not advice.

A Note on Scope & Mock Data

This is a one-week hackathon. We're not expecting you to build integrations with CRM systems, portfolio platforms, or provider APIs—that's months of work.

Instead, work with mock data that represents realistic advisor scenarios. We'll evaluate your approach to the problem, your architecture, and your thinking—not whether you've connected to Intelliflo.

Create sample client profiles, example suitability reports, realistic template variations. The mock data should feel authentic to the problems described below.

Challenge 01: Template Intelligence Engine

The Problem Space

The Template Setup Nightmare

When financial advice firms engage with AdvisoryAI to generate suitability reports or annual review reports, they don't arrive with a single template. They arrive with *dozens*.

Each template is a unique snowflake. Different logos. Different margins. Different fonts. Different table structures. Different section orders. Different compliance language. Different branding guidelines. Every firm believes their template is special—and they're right.

The typical process looks like this:

A new advisory firm signs up with five different report templates—one for pensions, one for investments, one for protection, one for annual reviews, one for ad-hoc advice. Each template is a Word document with carefully crafted formatting that took their compliance team months to approve.

Our delivery team receives these templates and begins the manual process: identifying which sections are static (headers, footers, legal disclaimers, compliance boilerplate) and which sections need to be dynamically generated by the LLM (client-specific recommendations, risk assessments, rationale explanations).

They insert placeholders. They write prompts. They test outputs. The formatting breaks. They fix the formatting. The tone is wrong. They adjust the prompts. The tables don't render correctly. They rebuild the tables.

Each template takes approximately 4 hours to set up. Multiply that by five templates per firm, and onboarding becomes a 20+ hour endeavour before a single report is generated.

The WordML challenge:

Behind every Word document lies WordML—Microsoft's XML-based format. To programmatically generate content into templates, we need to parse this structure, identify insertion points, and generate new XML that integrates seamlessly with existing styles.

Get it wrong and documents break. Tables misalign. Styles reset. The document opens with errors—or doesn't open at all. Current approaches rely on manual mapping: engineers inspect the XML, identify paths, and hard-code insertion logic. It works, but it doesn't scale.

The formatting reality:

Word documents are deceptively complex. A "simple" template might include: custom heading styles, tables with specific column widths, embedded logos, conditional sections, firm-specific numbered lists, variable footers, and draft watermarks.

When an LLM generates content into these templates, maintaining formatting is a constant battle. Text overflows. Tables break. Styles reset. The output looks "off" in ways that compliance teams immediately reject.

The prompt engineering burden:

Each variable section needs its own prompt engineering. A single suitability report might have 15-20 variable sections, each requiring custom prompts. Getting them right takes iteration. And when the firm requests a "small change" to their template, prompts that worked perfectly now produce awkward outputs.

Scale of the problem:

AdvisoryAI serves hundreds of advisory firms. Each firm has multiple templates. At 4 hours per template, onboarding takes weeks. Template modifications require engineering involvement. The delivery team is a bottleneck.

This doesn't scale. Every new firm is a significant onboarding burden. Every template change is a support ticket.

Context for Your Build

The template problem is fundamentally about understanding document structure, intelligent content generation, and seamless formatting preservation.

The goal: when we receive a template from a firm, setup should be near-instant. No 4-hour manual process. No weeks of onboarding. The system should intelligently parse the document, identify static vs. dynamic sections, map insertion points, and be ready to generate.

Your mock data should represent realistic template scenarios: suitability reports with different structures, annual reviews with varying sections, templates with tables, logos, and conditional content.

Think about what "automation" actually means here. Can a system learn to parse templates and identify insertion points automatically? Can formatting be preserved without manual intervention? Where are the boundaries?

Final Thought

The best financial advice is personalised, compliant, and clearly communicated. Templates are the vessel for that communication.

Your challenge: understand the template problem deeply, then show us your approach to solving it.

Good luck builders.